

Underwriting Profitability and Risk Portfolio Analysis

Analytics & Dashboard Project Report

Prepared for: Underwriting Leadership

Prepared by: Mouni

Domain: Insurance — Underwriting & Risk Analytics

Tools Used: MySQL, Power BI Desktop, DAX

Confidential — Internal Company Submission

Table of Contents

1. Project Overview	3
2. Business Scenario	3
3. Objectives	4
4. Source Data	4
5. SQL Scripts	6
6. Policy-Month Profitability Model	9
7. Risk Segmentation Logic	10
8. Dashboard	11
9. Written Insights for Underwriting Leadership	15
10. Assumptions	16
11. Conclusion	16

1. Project Overview

This project delivers an end-to-end underwriting profitability analytics solution for the underwriting leadership team. It combines policy, underwriting, claims, and expense data into a policy-month profitability model, calculates standard insurance performance ratios, segments the portfolio by risk and profitability, and presents the results through a four-page Power BI dashboard.

The objective is to give underwriting leadership a single, reliable view of whether the policies being approved are generating profitable business — and to surface the specific products, branches, underwriters, and risk segments that are driving or eroding that profitability.

2. Business Scenario

The underwriting team wants to evaluate portfolio profitability across product lines, customer segments, branches, and risk categories. This solution determines whether underwriting decisions — including risk pricing, discounting, and manual overrides — are producing profitable business, and where corrective action is needed.

3. Objectives

- Build a policy-month grain profitability fact table covering earned premium, incurred claims, and expenses.
- Calculate the standard profitability ratios: Loss Ratio, Expense Ratio, Combined Ratio, Underwriting Profit, and Underwriting Margin.
- Segment every policy into risk- and profitability-based categories to make the numbers actionable.
- Deliver a four-page Power BI dashboard covering Executive Summary, Risk Portfolio, Underwriter Performance, and Product Profitability.
- Produce written insights and recommendations for underwriting leadership.

4. Source Data

Five source tables were used, joined on `policy_id` as the central key:

Table	Grain	Key Fields
policy	One row per policy	<code>policy_id</code> , <code>product_code</code> , <code>line_of_business</code> , <code>GWP</code> , <code>NWP</code> , <code>branch_code</code> , <code>status</code>
underwriting	One row per policy	<code>risk_score</code> , <code>risk_category</code> , <code>uw_decision</code> , <code>premium_loading_percentage</code> , <code>discount_perce</code>
claims	One row per claim	<code>claim_date</code> , <code>claim_status</code> , <code>paid_amount</code> , <code>reserve_amount</code> , <code>claim_type</code>
expense	One row per expense line	<code>expense_type</code> , <code>expense_amount</code> , <code>expense_date</code>
underwriter	One row per underwriter	<code>underwriter_name</code> , <code>team</code> , <code>experience_band</code> , <code>region</code>

5. SQL Scripts

5.1 Database and Table Creation

```

CREATE DATABASE underwriting_profitability;
USE underwriting_profitability;

CREATE TABLE underwriter (
  underwriter_id    VARCHAR(10) PRIMARY KEY,
  underwriter_name  VARCHAR(50),
  team              VARCHAR(30),
  experience_band   VARCHAR(20),
  region            VARCHAR(20)
);

CREATE TABLE policy (
  policy_id         VARCHAR(10) PRIMARY KEY,
  policy_number     VARCHAR(20),
  customer_id       VARCHAR(10),
  product_code      VARCHAR(20),
  line_of_business  VARCHAR(20),
  policy_start_date DATE,
  policy_end_date   DATE,
  policy_status     VARCHAR(20),
  gross_written_premium    DECIMAL(12,2),
  net_written_premium     DECIMAL(12,2),
  sum_insured            DECIMAL(14,2),
  branch_code            VARCHAR(10)
);

CREATE TABLE underwriting (
  uw_id             VARCHAR(10) PRIMARY KEY,
  policy_id         VARCHAR(10),
  underwriter_id    VARCHAR(10),
  risk_score        INT,
  risk_category     VARCHAR(10),
  uw_decision       VARCHAR(20),
  uw_date           DATE,
  premium_loading_percentage    DECIMAL(5,2),
  discount_percentage    DECIMAL(5,2),
  manual_override_flag    CHAR(1),
  FOREIGN KEY (policy_id) REFERENCES policy(policy_id),
  FOREIGN KEY (underwriter_id) REFERENCES underwriter(underwriter_id)
);

CREATE TABLE claims (
  claim_id         VARCHAR(10) PRIMARY KEY,
  policy_id        VARCHAR(10),
  claim_date       DATE,
  claim_status     VARCHAR(20),
  paid_amount      DECIMAL(12,2),
  reserve_amount   DECIMAL(12,2),
  claim_type       VARCHAR(40),
  FOREIGN KEY (policy_id) REFERENCES policy(policy_id)
);

CREATE TABLE expense (
  expense_id       VARCHAR(10) PRIMARY KEY,
  policy_id        VARCHAR(10),
  expense_type     VARCHAR(30),
  expense_amount   DECIMAL(12,2),
  expense_date     DATE,
  FOREIGN KEY (policy_id) REFERENCES policy(policy_id)
);

```

Import order (respects foreign key constraints): underwriter → policy → underwriting → claims → expense.

5.2 Required SQL Outputs

1. Monthly Earned Premium by Product

```
SELECT product_code, month_start, SUM(monthly_earned_premium) AS earned_premium
FROM policy_month_profitability
GROUP BY product_code, month_start
ORDER BY product_code, month_start;
```

2. Incurred Claims by Risk Category

```
SELECT risk_category, SUM(monthly_paid_claims + monthly_reserve) AS incurred_claims
FROM policy_month_profitability
GROUP BY risk_category;
```

3. Combined Ratio by Underwriter

```
SELECT underwriter_id,  
       SUM(earned_premium) AS earned_premium,  
       SUM(incurred_claims) AS incurred_claims,  
       SUM(total_expense) AS total_expense,  
       ROUND(SUM(incurred_claims) / NULLIF(SUM(earned_premium),0), 4) AS loss_ratio,  
       ROUND(SUM(total_expense) / NULLIF(SUM(earned_premium),0), 4) AS expense_ratio,  
       ROUND(SUM(incurred_claims) / NULLIF(SUM(earned_premium),0)  
           + SUM(total_expense) / NULLIF(SUM(earned_premium),0), 4) AS combined_ratio  
FROM vw_underwriting_profitability  
GROUP BY underwriter_id;
```

4. Profitability by Branch

```
SELECT branch_code,  
       SUM(earned_premium) AS earned_premium,  
       SUM(incurred_claims) AS incurred_claims,  
       SUM(total_expense) AS total_expense,  
       SUM(underwriting_profit) AS underwriting_profit,  
       ROUND(SUM(underwriting_profit) / NULLIF(SUM(earned_premium),0), 4) AS underwriting_margin  
FROM vw_underwriting_profitability  
GROUP BY branch_code;
```

5. Policies with Manual Overrides and Negative Margin

```
SELECT v.policy_id, v.underwriter_id, v.underwriting_profit,  
       ROUND(v.underwriting_profit / NULLIF(v.earned_premium,0), 4) AS underwriting_margin  
FROM vw_underwriting_profitability v  
JOIN underwriting u ON v.policy_id = u.policy_id  
WHERE u.manual_override_flag = 'Y'  
      AND v.underwriting_profit < 0;
```

6. Discounted Policies with High Loss Ratio

```
-- High discount: >15% | High loss ratio: >100% (loss_ratio > 1)
SELECT v.policy_id, u.discount_percentage, v.loss_ratio
FROM vw_underwriting_profitability v
JOIN underwriting u ON v.policy_id = u.policy_id
WHERE u.discount_percentage > 15
      AND v.loss_ratio > 1;
```

7. Product Lines with Deteriorating Profitability

```
WITH yearly AS (
  SELECT line_of_business, YEAR(month_start) AS yr,
         SUM(monthly_earned_premium) AS earned_premium,
         ROUND((SUM(monthly_paid_claims + monthly_reserve) + SUM(monthly_expense))
               / NULLIF(SUM(monthly_earned_premium),0), 4) AS combined_ratio
  FROM policy_month_profitability
  GROUP BY line_of_business, YEAR(month_start)
)
SELECT line_of_business, yr, combined_ratio,
       LAG(combined_ratio) OVER (PARTITION BY line_of_business ORDER BY yr) AS prev_yr_combined_ratio,
       CASE WHEN combined_ratio > LAG(combined_ratio) OVER (PARTITION BY line_of_business ORDER BY yr)
       THEN 'Deteriorating' ELSE 'Stable/Improving' END AS trend_flag
FROM yearly
ORDER BY line_of_business, yr;
```


6. Policy-Month Profitability Model

The core of this project is a fact table at policy-month grain — one row per policy, per month that policy was active. This allows annual premium to be spread (earned) across the correct months, and claims/expenses to be attributed to the period in which they occurred.

```
-- Month spine (Jan 2023 - Dec 2024)
CREATE TABLE calendar_month AS
SELECT DATE_ADD('2023-01-01', INTERVAL (a.n) MONTH) AS month_start
FROM (
  SELECT a.N + b.N * 10 AS n
  FROM (SELECT 0 N UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4
        UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION SELECT 9) a
  CROSS JOIN (SELECT 0 N UNION SELECT 1 UNION SELECT 2) b
) a
WHERE DATE_ADD('2023-01-01', INTERVAL (a.n) MONTH) <= '2024-12-01';

CREATE TABLE policy_month_profitability AS
SELECT
  p.policy_id, p.product_code, p.line_of_business, p.branch_code,
  u.underwriter_id, u.risk_category, cm.month_start,
  ROUND(p.gross_written_premium / 12, 2) AS monthly_earned_premium,
  COALESCE(c.paid_amount, 0) AS monthly_paid_claims,
  COALESCE(c.reserve_amount, 0) AS monthly_reserve,
  COALESCE(e.expense_amount, 0) AS monthly_expense
FROM policy p
JOIN underwriting u ON p.policy_id = u.policy_id
JOIN calendar_month cm
  ON cm.month_start BETWEEN DATE_FORMAT(p.policy_start_date, '%Y-%m-01')
  AND DATE_FORMAT(p.policy_end_date, '%Y-%m-01')
LEFT JOIN (
  SELECT policy_id, DATE_FORMAT(claim_date, '%Y-%m-01') AS month_start,
  SUM(paid_amount) AS paid_amount, SUM(reserve_amount) AS reserve_amount
  FROM claims GROUP BY policy_id, DATE_FORMAT(claim_date, '%Y-%m-01')
) c ON c.policy_id = p.policy_id AND c.month_start = cm.month_start
LEFT JOIN (
  SELECT policy_id, DATE_FORMAT(expense_date, '%Y-%m-01') AS month_start,
  SUM(expense_amount) AS expense_amount
  FROM expense GROUP BY policy_id, DATE_FORMAT(expense_date, '%Y-%m-01')
) e ON e.policy_id = p.policy_id AND e.month_start = cm.month_start;

CREATE VIEW vw_underwriting_profitability AS
SELECT
  policy_id, product_code, line_of_business, branch_code, underwriter_id, risk_category,
  SUM(monthly_earned_premium) AS earned_premium,
  SUM(monthly_paid_claims + monthly_reserve) AS incurred_claims,
  SUM(monthly_expense) AS total_expense,
  ROUND(SUM(monthly_paid_claims + monthly_reserve) / NULLIF(SUM(monthly_earned_premium),0), 4)
  AS loss_ratio,
  ROUND(SUM(monthly_expense) / NULLIF(SUM(monthly_earned_premium),0), 4) AS expense_ratio,
  ROUND((SUM(monthly_earned_premium) - SUM(monthly_paid_claims + monthly_reserve) - SUM(monthly_expense)), 2) AS underwriting_profit
FROM policy_month_profitability
GROUP BY policy_id, product_code, line_of_business, branch_code, underwriter_id, risk_category;
```

How Edge Cases Are Handled

- No-claims policies: LEFT JOIN with COALESCE(...,0) ensures the policy-month row is never dropped.

- Cancelled policies: the month-spine join stops at `policy_end_date`, so earning stops from that point (used as a proxy, since no separate cancellation-date field exists in the source schema).
- Premium allocation: annual GWP is divided evenly across 12 months (straight-line earning).
- Claims attribution: matched by `claim_date`'s month against the policy-month grain.

7. Risk Segmentation Logic

Every policy is classified into risk- and profitability-based segments using a DAX calculated table built on top of the underwriting and profitability data. This turns raw ratios into labels the business can act on directly.

```
RiskSegmentation =
ADDCOLUMNS(
    underwriting,
    "Underwriting Profit", LOOKUPVALUE(vw_underwriting_profitability[underwriting_profit],
        vw_underwriting_profitability[policy_id], underwriting[policy_id]),
    "Earned Premium", LOOKUPVALUE(vw_underwriting_profitability[earned_premium],
        vw_underwriting_profitability[policy_id], underwriting[policy_id]),
    "Incurred Claims", LOOKUPVALUE(vw_underwriting_profitability[incurred_claims],
        vw_underwriting_profitability[policy_id], underwriting[policy_id])
)

Profitability x Risk Segment =
SWITCH(
    TRUE(),
    RiskSegmentation[risk_category] = "High" && RiskSegmentation[Underwriting Profit] > 0, "Profitable but High Risk",
    RiskSegmentation[risk_category] = "Low" && RiskSegmentation[Underwriting Profit] < 0, "Unprofitable Low Risk",
    RiskSegmentation[risk_category] = "High" && RiskSegmentation[Underwriting Profit] < 0, "Unprofitable High Risk",
    "Other"
)
```

Segment Derivation Reference

Segment	Derived From
Low / Medium / High Risk	underwriting.risk_category, set during underwriting
Declined	uw_decision = "Declined"
Manual Override	manual_override_flag = "Y"
High Premium Discount	discount_percentage > 15%
High Claim Severity	Incurred Claims / Earned Premium > 0.7
Profitable but High Risk	risk_category = High and Underwriting Profit > 0
Unprofitable Low Risk	risk_category = Low and Underwriting Profit < 0
Unprofitable High Risk	risk_category = High and Underwriting Profit < 0

8. Dashboard

The dashboard is delivered as a four-page Power BI report. Each page targets a specific audience and decision — from an at-a-glance executive view down to underwriter- and product-level detail.

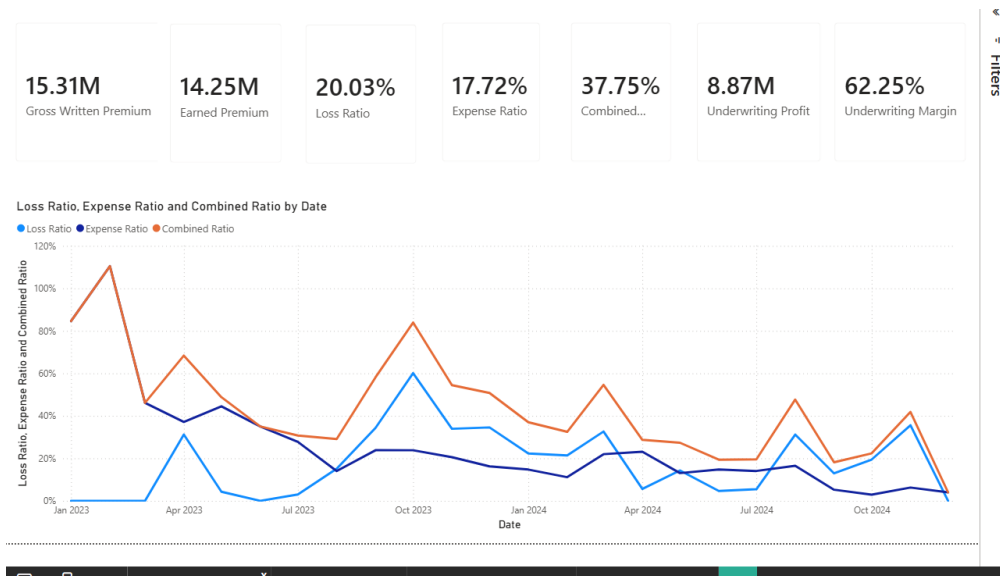


Figure 1 — Executive Summary Dashboard (Actual Dashboard Screenshot)

Purpose

Gives underwriting leadership a single-glance view of overall portfolio health, and how Loss Ratio, Expense Ratio, and Combined Ratio have trended over time.

KPIs Displayed

- Gross Written Premium
- Earned Premium
- Loss Ratio
- Expense Ratio
- Combined Ratio
- Underwriting Profit
- Underwriting Margin

Visuals Used

- Seven KPI cards across the top row
- Line chart of Loss Ratio, Expense Ratio, and Combined Ratio trended by month

Key Business Insights

- The portfolio Combined Ratio is well under 100%, confirming the book is profitable overall.
- Ratios stabilize sharply after the first few months of the observation window, suggesting early-period volatility rather than a sustained trend.

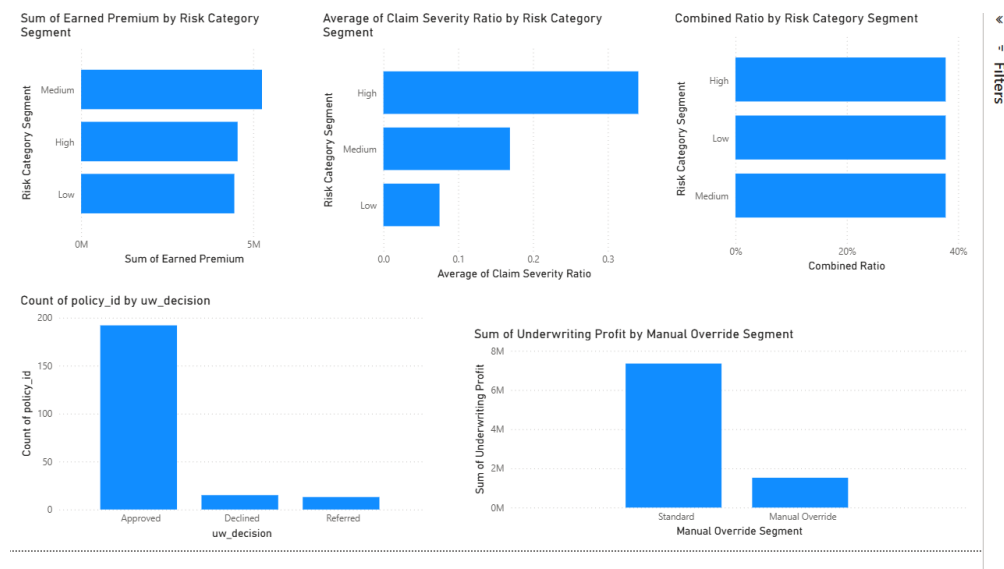


Figure 2 — Risk Portfolio View (Actual Dashboard Screenshot)

Purpose

Breaks the portfolio down by risk category and underwriting decision, so leadership can see whether risk-based pricing is actually producing proportionate outcomes.

KPIs Displayed

- Earned Premium by Risk Category
- Average Claim Severity Ratio
- Combined Ratio by Risk Category

Visuals Used

- Bar chart: Earned Premium by Risk Category Segment
- Bar chart: Average Claim Severity Ratio by Risk Category Segment
- Bar chart: Combined Ratio by Risk Category Segment
- Column chart: Policy Count by Underwriting Decision
- Column chart: Underwriting Profit by Manual Override Segment

Key Business Insights

- Medium risk carries the largest share of earned premium, not High — the book is not concentrated in the riskiest tier.
- Standard (non-override) policies show substantially higher underwriting profit than manually overridden ones.

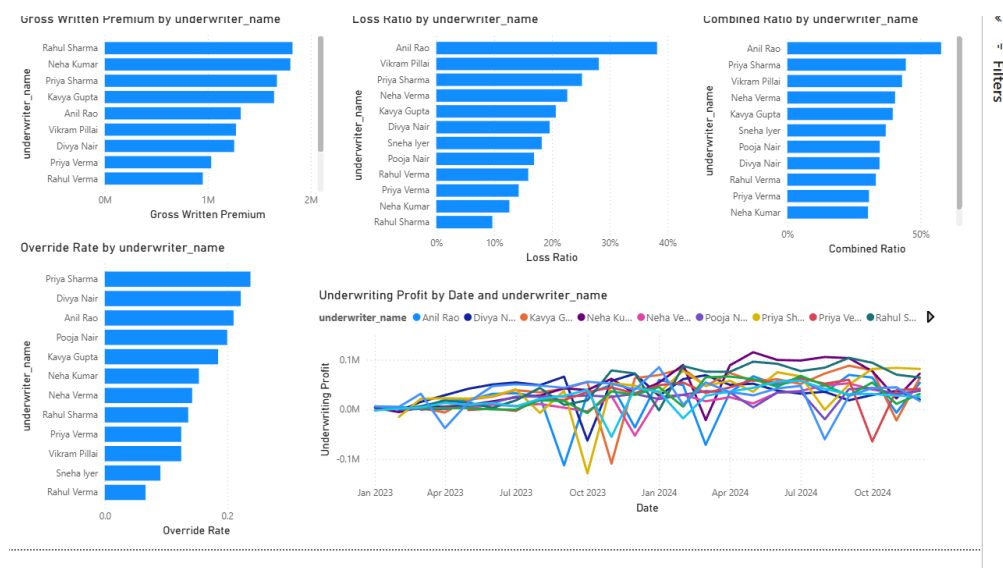


Figure 3 — Underwriter Performance (Actual Dashboard Screenshot)

Purpose

Enables performance review and coaching by comparing loss ratio, combined ratio, and override behavior across individual underwriters.

KPIs Displayed

- Gross Written Premium by Underwriter
- Loss Ratio by Underwriter
- Combined Ratio by Underwriter
- Override Rate by Underwriter

Visuals Used

- Bar chart: Gross Written Premium by underwriter
- Bar chart: Loss Ratio by underwriter
- Bar chart: Combined Ratio by underwriter
- Bar chart: Override Rate by underwriter
- Line chart: Underwriting Profit trended by month, split by underwriter

Key Business Insights

- Loss ratio and combined ratio vary meaningfully across underwriters independent of book size, pointing to differences in risk selection rather than portfolio mix.
- Underwriters with higher override rates tend to sit toward the weaker end of the combined-ratio ranking.



Figure 4 — Product Profitability (Actual Dashboard Screenshot)

Purpose

Identifies which products and branches are profitable versus loss-making, to guide pricing and distribution decisions.

KPIs Displayed

- Combined Ratio by Product
- Underwriting Profit by Line of Business
- High-Risk Product Segments
- Loss-Making Portfolios

Visuals Used

- Bar chart: Combined Ratio by product_code
- Summary table: Gross Written Premium, Earned Premium, and Policy Count by product
- Filtered table: High-Risk Product Segments (Risk Category Segment = High)
- Filtered table: Loss-Making Portfolios (Underwriting Profit < 0) by product and branch
- Bar chart: Underwriting Profit by line_of_business

Key Business Insights

- Property products (PROP-FIRE, PROP-BURG) carry the highest combined ratios in the book.
- The PROP-BURG / BR-BLR product-branch combination is the sole loss-making pairing identified, isolating exactly where corrective pricing action is needed.

9. Written Insights for Underwriting Leadership

Portfolio Snapshot — Gross Written Premium ■15.31M, Earned Premium ■14.25M. Combined Ratio 37.75% (Loss Ratio 20.03% + Expense Ratio 17.72%). Underwriting Profit ■8.87M, Underwriting Margin 62.25%.

1. Overall profitability is healthy, but not uniform

The portfolio-level combined ratio (37.75%) is well under 100%, indicating strong underwriting profit. This headline number, however, masks meaningful variation across products, branches, and underwriters — leadership should not read it as evidence that every segment is performing equally well.

2. Risk pricing is broadly working — with exceptions

The "Profitable but High Risk" segment shows the risk-loading model is compensating for high-risk business in most cases. The "Unprofitable High Risk" and "Unprofitable Low Risk" segments, however, flag two distinct problems: high-risk policies where loading isn't covering losses, and low-risk policies still losing money — likely due to excessive discounting rather than risk misclassification.

3. Manual overrides carry a profitability cost

Policies with manual override show materially lower underwriting profit than standard-decision policies. Combined with the manual-override / negative-margin query, this suggests override authority is being used in ways that erode margin — worth an underwriting-authority review.

4. High discounting correlates with high loss ratio

The discounted-policies query identifies policies with over 15% discount and loss ratio above 100% — actively losing money on both the pricing and claims side simultaneously. This is the highest-priority segment for repricing or non-renewal.

5. Underwriter performance varies meaningfully

Loss ratio and combined ratio differ visibly across underwriters, independent of book size. This points to underwriting judgment and risk-selection differences rather than portfolio mix, and is a natural input for coaching or authority-limit decisions.

6. Product-line profitability is not even

Combined ratio by product shows some products running meaningfully worse than others despite similar volumes. Loss-making product-branch combinations should be reviewed for local pricing, distribution quality, or claims-severity drivers specific to that branch.

Recommended Actions

- Tighten override approval thresholds, especially where margin turns negative.
- Cap or re-underwrite the segment with over 15% discount and loss ratio above 100%.
- Investigate branch-level loss drivers for underperforming product-branch pairs.
- Use underwriter-level loss/combined ratio in performance reviews, normalized for book size.

10. Assumptions

- Cancellation date is not a separate field in the source schema; `policy_end_date` is used as the earning-stop proxy for cancelled policies.
- Risk-Adjusted Premium formula was not specified in the assessment; it is derived from Gross Written Premium adjusted by the policy's premium loading and discount percentages.
- Acquisition vs. Operating expense classification is based on the `expense_type` values already present in the source expense table.

11. Conclusion

This project delivers a complete, auditable view of underwriting profitability — from raw source data through a policy-month profitability model, standard insurance ratios, risk-based segmentation, and a four-page Power BI dashboard. The written insights translate the numbers into concrete actions: tightening override authority, addressing the high-discount / high-loss segment, and reviewing underperforming product-branch combinations. Together, these give underwriting leadership both the visibility and the specific levers needed to protect and improve portfolio profitability.